ASSIGNMENT 3

— Build Bug Report Management Application with User Authentication —

1. Overview

In this assignment, you will continue exploring user **authentication**. We have already set up the REST API server to validate an ordinary user. The requirements are still evolving from the **Bug Report project**.

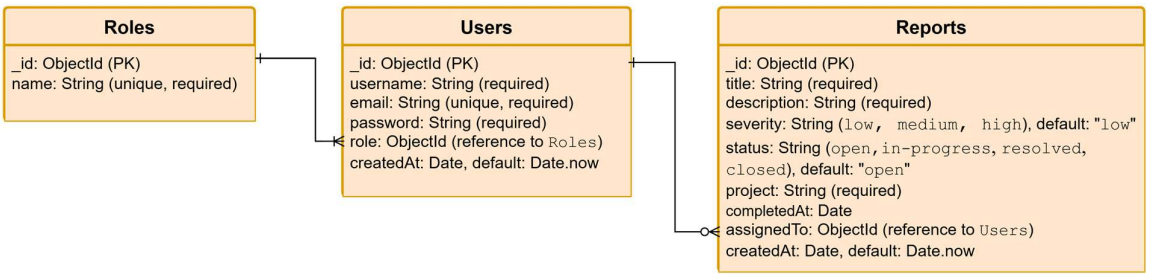
The system uses **JWT authentication** and provides RESTful API endpoints.

• **Main Technologies**: Node.js, Express.js, Mongoose, JWT, bcrypt, dotenv, jsonwebtoken

• **Tools**: Postman, MongoDB Compass, Visual Studio Code.

2. Main requirements:

Data Structure:



We will build a Node.js Express application in that:

- User can signup new account, or login with email & password.
- By role (admin, moderator, user), the User has access to protected resources or not.

Project Structure:



Task 1: Project Initialization and Configuration **(2.0 points)**

- Initialize Node.js project named: **assignment3_<memberCode>** (lowercase). (ex: assignment3_sangnqce150621) (1 points)
- Create a database named **BugReport** and define the required collections and their data structures. (See **Figure 1)**. (0.5 points)
- Create a **.env** file to define essential environment variables, including the server port (**PORT**), MongoDB connection string (**MONGO_URI**), and JWT secret (**JWT_SECRET**). This helps your application load configuration securely and ensure that the server runs on http://localhost:8002. (0.5 points)

Task 2: API – POST /users/signup **(2.0 points)**

- Create a POST API endpoint that allows a new user to register by sending **username, email, password and role** in the request body. Return a JSON message confirming successful registration. If the email does not exist or the password is incorrect, return an appropriate error message with HTTP status code 401. (1.5 points)
- The server must hash the password using **bcrypt** before storing it in the users collection to ensure security. (0.5 points)

Task 3: API – POST /users/signin **(2.0 points)**

- Create a POST endpoint at **/users/signin** that allows users to log in with email and password.
- This API must validate the credentials by comparing the password with the hashed version in the database and return a signed **JWT token** in JSON format if authentication succeeds. If invalid, return an appropriate error message

Task 4: API – GET /reports **(2.0 points)**

- Create a GET endpoint at /reports that retrieves all bug reports assigned to the currently authenticated user.
- The API must authenticate the user using a JWT token. The token must be sent via the Authorization header using the Bearer scheme.
- The system must retrieve only the reports whose assignedTo field matches the ID of the authenticated user.
- The system must use Mongoose populate() to populate the assignedTo field with the user's ***_id, username, and email***.
- The response must be returned in JSON format.
- Users must not be able to retrieve reports assigned to other users.

Task 5: API – POST /reports **(2.0 points)**

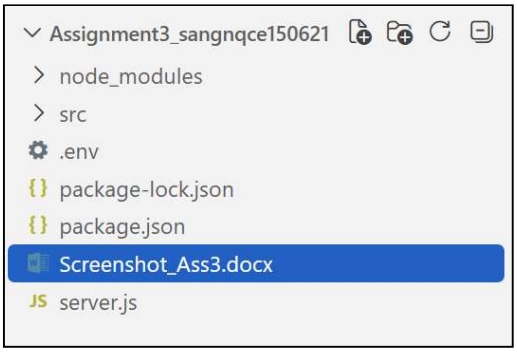
- Create a POST endpoint at /reports that allows an authenticated Admin to create a new bug report.
- The API must authenticate the user using a JWT token. The token must be sent via the Authorization header using the Bearer scheme.
- The system must verify that the authenticated user's role is "admin". If the user is not an Admin, return HTTP status code 403 Forbidden.
- The request body must include:
  - *title*
  - *description*
  - *severity*
  - *project*
  - *assignedTo (the ID of the user assigned to handle the report)*

- The system must verify that the specified **assignedTo** user exists and has the role "user". Reports must not be assigned to Admins or Moderators.
- The newly created report must have the default status "open".
- If the report is successfully created, return an appropriate JSON response containing a success message and the created report..

3. Submission Requirements:

a) Source Code Folder: Include full project (remove node_modules) with comments at the top of each file.

b) Documentation File (.docx): Name the file Screenshot_Ass3.docx. Download the file, open it, and paste the screenshot as instructed into the correct location. Total of 14 screenshots. After you're done, copy it to the root project folder before submitting your assignment.



c) Submission Format: Submit as .zip/.rar named: assignment3_<memberCode>.zip

Note: Please use the attached resource.zip file as sample data and test cases for Postman.